毕业论文第 2 章　问题描述与数学模型（草稿）

FJSP-AGV-RL Research Platform

2026-11-01

草稿说明　本文是论文第 2 章的草稿，对应仓库当前实现（`src/fjsp_lab/maintenance/`）。模型假设来自用户确认的毕设首版设定，见 `docs/models/graduation_model.md`。各结论的状态如下：

- 命题 2.3、引理 2.5、推论 2.6、命题 2.8 在本文给出完整证明；
- 定理 2.7 的归约由本文推导，所引用的经典结果见参考文献，**需导师确认**；
- 注 2.9 所述的解码器构造空间能否覆盖最优解**没有证明**，只在小规模算例上穷举验证过。

第二章　问题描述与数学模型

2.1　问题描述

考虑由 m 台同质平行机组成的车间，需要加工 n 个工件。每个工件只有一道工序，可以在任意一台机器上加工，加工时间与机器无关；所有工件在 0 时刻到达，各有一个交货期。每台机器有若干个预先给定、按时间排列且互不重叠的预防维护时间窗；每个时间窗内必须完整地做一次维护，维护期间该机器不能加工。即使某台机器没有分到任何工件，它的维护也必须执行。目标是最小化总延期。

调度需要同时决定三件事：工件分配到哪台机器（任务分配）、每台机器上工件的先后（加工顺序）、每次维护的开始时刻（维护时机）。本文比较两种维护方式：

- **固定维护**：每次维护的开始时刻预先给定，调度只决定任务分配和加工顺序；
- **联合维护**（协同优化）：维护开始时刻也作为决策变量，在时间窗内自由选择。

每台机器只有一个时间窗时，称为**单窗口模型**，它对应 Lee 与 Chen [1] 研究的“每台机器在给定期限内恰好维护一次、不同机器可同时维护”的情形；本文把期限 $[0,T]$ 推广为一般时间窗 $[L,U)$，目标由加权总完工时间改为总延期，并增加了固定与联合两种维护方式的对比。允许每台机器有多个时间窗、且各机器窗口数可以不同时，称为**周期性多窗口模型**。单窗口模型是多窗口模型的特例，下文统一按多窗口模型叙述。

模型采用以下假设：

(A1) 机器同质，工件 j 在任一机器上的加工时间都是 p_j；

(A2) 每个工件一道工序，加工不可中断，工件之间没有先后约束，全部在 0 时刻到达；

(A3) 机器 i 的每个时间窗内必须恰好执行一次维护，维护时长给定，维护不能省略；

(A4) 整个维护区间必须落在时间窗内，而不是只约束维护的开始时刻；

(A5) 维护与该机器上的加工互斥；不同机器的维护可以同时进行，不考虑维修人员数量限制；

(A6) 不考虑运输、换型准备、随机故障和抢占；所有时间为非负整数。

2.2　符号

本章符号见表 2.1。输入数据满足

$$0 \le L_{ik}, \qquad q_{ik} \le U_{ik} - L_{ik}, \tag{2.1}$$

$$U_{ik} \le L_{i,k+1} \quad (k < K_i), \tag{2.2}$$

$$L_{ik} \le f_{ik} \le U_{ik} - q_{ik} \quad \text{（仅固定维护方式）}. \tag{2.3}$$

表 2.1: 集合、参数与决策变量

符号	含义
$\mathcal{J}=\{1,\dots,n\}$	工件集合，$n\ge 1$
$\mathcal{I}=\{1,\dots,m\}$	机器集合，$m\ge 1$
$\mathcal{K}_i=\{1,\dots,K_i\}$	机器 i 的维护时间窗集合，$K_i\ge 1$
$W=\sum_i K_i$	时间窗（即维护）总数
$p_j\in\mathbb{Z}_{\ge 1}$	工件 j 的加工时间
$d_j\in\mathbb{Z}_{\ge 0}$	工件 j 的交货期
$q_{ik}\in\mathbb{Z}_{\ge 1}$	机器 i 第 k 次维护的时长
$[L_{ik},U_{ik})$	机器 i 第 k 个维护时间窗（左闭右开）
f_{ik}	固定维护方式下预先给定的维护开始时刻
$P=\sum_j p_j$	总加工时间
$U^*=\max_{i,k}U_{ik}$	最晚的时间窗右端点
$H=U^*+P$	时域上界（见推论 2.6）
$x_{ji}\in\{0,1\}$	工件 j 是否分配到机器 i
$S_j,\ C_j$	工件 j 的开工与完工时刻，$C_j=S_j+p_j$
M_{ik}	机器 i 第 k 次维护的开始时刻
T_j	工件 j 的延期，$T_j=\max(0,\,C_j-d_j)$

式 (2.1) 保证每次维护都能放进其时间窗，式 (2.2) 表示同一机器的时间窗按时间排列且互不重叠，式 (2.3) 保证预先给定的维护时刻本身合法。加工与维护都占用左闭右开区间：工件 j 占用 $[S_j,C_j)$，维护占用 $[M_{ik},M_{ik}+q_{ik})$。两个区间 $[a,b)$ 与 $[c,e)$ 不重叠当且仅当 $b\le c$ 或 $e\le a$。

2.3　数学模型

联合维护方式下的模型记为 (JM)：

$$\min \quad \sum_{j\in\mathcal{J}} T_j \tag{2.4}$$

$$\text{s.t.} \quad \sum_{i\in\mathcal{I}} x_{ji}=1, \qquad j\in\mathcal{J}, \tag{2.5}$$

$$C_j=S_j+p_j,\quad S_j\ge 0, \qquad j\in\mathcal{J}, \tag{2.6}$$

$$T_j=\max(0,\,C_j-d_j), \qquad j\in\mathcal{J}, \tag{2.7}$$

$$L_{ik}\le M_{ik}\le U_{ik}-q_{ik}, \qquad i\in\mathcal{I},\ k\in\mathcal{K}_i, \tag{2.8}$$

$$x_{ji}=x_{li}=1\ \Rightarrow\ C_j\le S_l\ \vee\ C_l\le S_j, \qquad j\ne l,\ i\in\mathcal{I}, \tag{2.9}$$

$$x_{ji}=1\ \Rightarrow\ C_j\le M_{ik}\ \vee\ M_{ik}+q_{ik}\le S_j, \qquad j\in\mathcal{J},\ i\in\mathcal{I},\ k\in\mathcal{K}_i, \tag{2.10}$$

$$x_{ji}\in\{0,1\},\quad S_j,\,M_{ik}\in\mathbb{Z}. \tag{2.11}$$

式 (2.5) 要求每个工件恰好分配到一台机器；式 (2.6) 表示加工不可中断；式 (2.7) 是延期的精确定义，提前完工不产生负延期，也不抵消其他工件的延期；式 (2.8) 要求整个维护区间落在时间窗内，对每个时间窗都必须成立，因此维护不能省略，空闲机器也要维护；式 (2.9) 和式 (2.10) 分别是同一机器上加工与加工、加工与维护的不重叠约束。

同一机器上两次维护之间不需要单独的约束：由式 (2.8) 和式 (2.2)，$M_{ik} + q_{ik} \le U_{ik} \le L_{i,k+1} \le M_{i,k+1}$，两次维护自然不重叠，并且按窗口顺序执行。

固定维护方式下的模型记为 (FM)，它在 (JM) 的基础上增加

$$M_{ik} = f_{ik}, \qquad i \in \mathcal{I},\ k \in \mathcal{K}_i. \tag{2.12}$$

注 2.1. 目标 (2.4) 是**正则目标**：每个 T_j 是 C_j 的非减函数，因此任何工件提前完工都不会使目标变差。引理 2.5 和命题 2.8 都依赖这一点；若改为提前也受罚、按时段计费等非正则目标，相关结论需要重新论证。

注 2.2. 析取约束 (2.9)、(2.10) 可以按常规方式引入排序 0–1 变量并以 H 为大 M 线性化，得到混合整数线性规划；由推论 2.6，限制在 $[0, H]$ 内不损失最优值。本文的精确求解采用第 2.6 节的约束规划模型，不使用线性化形式。

2.4　固定维护与联合维护的关系

命题 2.3. 记 $\mathcal{F}_{\mathrm{FM}}$、$\mathcal{F}_{\mathrm{JM}}$ 分别为 (FM)、(JM) 的可行解集，z^*_{FM}、z^*_{JM} 为相应最优值。在相同数据下 $\mathcal{F}_{\mathrm{FM}} \subseteq \mathcal{F}_{\mathrm{JM}}$，从而 $z^*_{\mathrm{JM}} \le z^*_{\mathrm{FM}}$。

证明. (FM) 的约束是 (JM) 的约束加上式 (2.12)，而由式 (2.3)，式 (2.12) 给出的维护时刻满足式 (2.8)。所以 (FM) 的每个可行解都是 (JM) 的可行解，两者目标函数相同，故联合最优值不大于固定最优值。两个最优值都存在，见推论 2.6。 □

该关系只对**两者都求得最优**的情形成立。若某一方只得到限时可行解，两个数值可能逆序，不能据此认为模型违背该命题，也不能把限时可行解称为最优解。实验中两种方式使用同一组数据、目标和生产约束，唯一区别是是否放开维护时机，因此二者之差衡量的就是协同决策维护时机带来的收益。

2.5　模型性质

2.5.1　左移解与时域上界

在可行解中，把同一机器上的工件和维护统称为该机器的**事件**。由不重叠约束，同一机器上的事件可以按开始时刻排成一个序列。

定义 2.4 (左移解)**.** 称可行解为**左移解**，如果每台机器上的每个事件都尽可能早地开始，即（记 e^- 为同一机器上的前一个事件，首个事件取 $C_{e^-} = 0$）：

- 工件 j：$S_j = C_{e^-}$；
- 维护 (i,k)：联合方式下 $M_{ik} = \max(C_{e^-}, L_{ik})$，固定方式下 $M_{ik} = f_{ik}$。

引理 2.5 (左移占优)**.** 对 (JM) 或 (FM) 的任一可行解 σ，保持每台机器上的事件集合与先后顺序不变，按定义 2.4 依次重新确定开始时刻，得到的 σ' 是同一模型的可行解，且对每个事件，σ' 中的开始时刻不晚于 σ 中的开始时刻。特别地，$C'_j \le C_j$ 对所有 j 成立，从而 $\sum_j T'_j \le \sum_j T_j$。

证明. 固定一台机器 i，设其事件序列为 $e_1, e_2, \ldots$，对位置 r 归纳证明 $\mathrm{start}'(e_r) \le \mathrm{start}(e_r)$，从而 $\mathrm{end}'(e_r) \le \mathrm{end}(e_r)$。记 $c = \mathrm{end}(e_{r-1})$、$c' = \mathrm{end}'(e_{r-1})$（$r = 1$ 时均为 0），由归纳假设 $c' \le c$；又因 σ 中事件不重叠，$c \le \mathrm{start}(e_r)$。

- e_r 是工件：$\mathrm{start}'(e_r) = c' \le c \le \mathrm{start}(e_r)$。
- e_r 是联合方式下的维护 (i,k)：σ 满足 $M_{ik} \ge L_{ik}$，故 $\max(c', L_{ik}) \le \max(c, L_{ik}) \le M_{ik}$。
- e_r 是固定方式下的维护：$\mathrm{start}'(e_r) = f_{ik} = \mathrm{start}(e_r)$。

可行性：每个事件都在前一事件结束后开始（固定方式下 $c' \le c \le f_{ik}$），所以同一机器上不重叠，式 (2.9)、(2.10) 成立；联合方式下 $M'_{ik} \ge L_{ik}$ 且 $M'_{ik} + q_{ik} \le M_{ik} + q_{ik} \le U_{ik}$，固定方式下 $M'_{ik} = f_{ik}$，故式 (2.8)、(2.12) 成立；分配与时长不变，式 (2.5)、(2.6) 成立。最后，T_j 是 C_j 的非减函数，所以总延期不增加。 □

推论 2.6 (时域上界)**.** (JM) 与 (FM) 都有可行解和最优解；并且存在一个最优解，其所有 S_j、C_j、M_{ik}、T_j 都在 $[0, H]$ 内，其中 $H = U^* + P$。因此把这些变量的取值范围限制为 $[0, H]$ 不会损失最优值。

证明. *可行解存在*：联合方式取 $M_{ik} = L_{ik}$，固定方式取 $M_{ik} = f_{ik}$；由式 (2.1)–(2.3) 这些维护合法且互不重叠，且都在 U^* 前结束。把全部工件依次排在机器 1 的最后一次维护之后，即得到可行解。

上界：设 σ' 为左移解。在机器 i 上，最后一次维护 (i, K_i) 之前的事件都在 $M'_{iK_i} \le U_{iK_i} - q_{iK_i} < U^*$ 前结束；这次维护在 $U_{iK_i} \le U^*$ 前结束；其后的工件首尾相接地加工，总时长不超过 P，所以最后完工不晚于 $U^* + P = H$。于是左移解的所有时刻都在 $[0, H]$ 内，且 $T_j \le C_j \le H$。

最优解存在：左移解由各机器的事件集合与顺序唯一确定，因此只有有限个。由引理 2.5，任一可行解都不优于某个左移解，所以在有限个左移解中取目标最小者，就得到全局最优解，并且它满足上述上界。 □

2.5.2　计算复杂性

记 (JM) 和 (FM) 的判定形式为：给定实例和整数 $y \ge 0$，是否存在可行解使 $\sum_j T_j \le y$。

定理 2.7. (JM) 与 (FM) 的判定形式都属于 NP，并且：

(a) 即使 $m = 1$、$K_1 = 1$，它们也是 NP 完全的；

(b) 当机器数 m 作为输入的一部分时，即使每台机器只有一个时间窗，它们也是强 NP 完全的。

单窗口模型是多窗口模型的特例，所以结论对两种模型都成立。

证明. *属于 NP*：由推论 2.6 的证明，若存在满足 $\sum_j T_j \le y$ 的可行解，则存在这样的左移解，其所有时刻不超过 H，编码长度是输入规模的多项式；按式 (2.5)–(2.12) 逐条验证只需多项式时间。

(a) 由单机总延期问题 $1\|\sum T_j$ 归约，该问题的判定形式是（普通意义下）NP 完全的 [2]。给定 $1\|\sum T_j$ 的实例 $(p_j,d_j)_{j\in\mathcal{J}}$ 与 y，构造：$m=1$，一个时间窗 $[P,P+1)$，$q=1$，固定方式取 $f=P$，阈值仍为 y。下面说明两个问题的最优值相等。

- 对 $1\|\sum T_j$ 的任一工件顺序，从 0 时刻起无空闲地加工，最后一个工件在 P 完工；再在 $[P,P+1)$ 维护，即得到两种方式下的可行解，各工件完工时刻不变。
- 反之，对构造实例的任一可行解，保持工件顺序，删去维护并从 0 时刻起无空闲地加工；每个工件的完工时刻等于其前面工件（含自身）的加工时间之和，不晚于原解中的完工时刻，所以总延期不增加。

因此构造实例存在 $\sum T_j\le y$ 的解当且仅当原实例存在，且构造是多项式的。

(b) 由 3-划分问题归约，该问题是强 NP 完全的 [3]：给定正整数 $a_1,\dots,a_{3t}$ 与 B，满足 $B/4<a_r<B/2$ 且 $\sum_r a_r=tB$，问能否把它们分成 t 组，使每组之和都等于 B（由取值范围，这样的每组恰有 3 个数）。构造：$m=t$ 台机器，$n=3t$ 个工件，$p_r=a_r$，$d_r=B$；每台机器一个时间窗 $[tB,\,tB+1)$，$q=1$，固定方式取 $f=tB$；阈值 $y=0$。

- 若存在 3-划分，把每组分到一台机器，从 0 时刻起无空闲地加工，每台机器在 $B\le tB$ 时刻完成；再在 $[tB,tB+1)$ 维护。所有 $C_r\le B=d_r$，总延期为 0。
- 若存在总延期为 0 的可行解，则所有 $C_r\le B$，每台机器上的工件都在 $[0,B)$ 内互不重叠地加工，负载不超过 B；而总负载为 tB，所以每台机器的负载恰为 B，即得到一个 3-划分。

构造实例中的数值都不超过 $tB+1$，是 3-划分实例一元编码长度的多项式，故归约保持强 NP 完全性。　□

定理 2.7 说明，除非 P = NP，本问题不存在多项式时间精确算法；机器数作为输入时，连伪多项式时间算法也不存在。这就是本文在大规模算例上采用调度规则、元启发式和强化学习，而只在小规模算例上用精确方法作参考的原因。文献 [1] 对每台机器在 $[0,T]$ 内维护一次、目标为总完工时间的情形证明了 NP 难，其模型与目标都和本文不同，所以本文给出了上面的独立归约。

2.5.3　构造式决策过程的完备性

第 3 章的调度构造过程建立在下面的逐机器追加方式之上。状态记录每台机器的当前结束时刻 τ_i（初值 0）、已安排的工件，以及每台机器下一个待做的时间窗 k_i。记机器 i 上“下一个工件最晚完工时刻”为

$$\lambda_i=\begin{cases}U_{ik_i}-q_{ik_i}, & \text{联合方式且机器 } i \text{ 还有待做窗口},\\ f_{ik_i}, & \text{固定方式且机器 } i \text{ 还有待做窗口},\\ +\infty, & \text{机器 } i \text{ 的窗口已全部完成}.\end{cases}$$

可选动作有两类：

- 工件动作 (j,i)：未安排的工件 j 在机器 i 的 τ_i 时刻开工，要求 $\tau_i+p_j\le\lambda_i$；
- 维护动作 i：机器 i 还有待做窗口时，在 $\max(\tau_i,L_{ik_i})$（联合）或 f_{ik_i}（固定）开始维护。

每个动作执行后更新 τ_i 和 k_i。所有工件和维护都安排完后，过程结束，共 $n+W$ 步。

命题 2.8. 对上述构造过程：

(i) 任何未结束的状态都至少有一个可选动作，每次构造都在 $n+W$ 步后得到一个可行解；

(ii) 每个左移解都能由某个动作序列构造出来。

因此，所有构造结果中的最优者就是 (JM)（或 (FM)）的最优解。

证明. (i) 先证不变式：机器 i 有待做窗口 k_i 时，$\tau_i \le \lambda_i$。初始 $\tau_i = 0 \le L_{i1} \le \lambda_i$；追加工件后由动作条件 $\tau_i \le \lambda_i$；完成维护 (i,k) 后 $\tau_i \le U_{ik} \le L_{i,k+1} \le \lambda_i$。由该不变式，有待做窗口的机器的维护动作总是合法的：联合方式下 $L_{ik_i} \le \max(\tau_i, L_{ik_i}) \le U_{ik_i} - q_{ik_i}$，固定方式下 $\tau_i \le f_{ik_i}$。若没有任何待做窗口，则所有 $\lambda_i = +\infty$，任一未安排工件都可以追加。所以未结束的状态总有可选动作；每步恰好安排一个工件或一次维护，故 $n+W$ 步后结束。每个事件都在本机器当前结束时刻或之后开始，同一机器上不重叠；维护落在时间窗内（固定方式下等于 f_{ik}）；所有工件和时间窗都已覆盖。因此结果是可行解。

(ii) 设 σ' 为左移解。把所有事件按任意一种保持各机器内部先后顺序的方式排成序列（例如按开始时刻），依次执行对应动作。对执行步数归纳，执行到机器 i 的事件 e 时，τ_i 等于 σ' 中 e 的前一事件的结束时刻（其他机器的动作不改变 τ_i）。

- e 是工件 j：动作在 $\tau_i = S'_j$ 开工，与 σ' 一致。若此时机器 i 还有待做窗口 k_i，则该维护在 σ' 中排在 j 之后，故 $C'_j \le M'_{ik_i} \le \lambda_i$，动作合法。
- e 是维护 (i,k)：由式 (2.2)，同一机器的维护按窗口顺序出现，所以此时 $k_i = k$；动作的开始时刻 $\max(\tau_i, L_{ik})$（或 f_{ik}）正是左移解的定义值。

所以动作序列合法，并且逐个复现 σ'。结合引理 2.5 与推论 2.6，存在左移的最优解，它可被构造出来；又由 (i)，构造结果都是可行解，所以构造结果中的最优值等于模型最优值。　□

注 2.9. 第 3 章的强化学习主方法使用的是另一种构造方式：在元启发式的序列解码器上逐个选择“下一个工件”或“某台机器的下一次维护”，而工件被放到使其**完工最早**的机器上。这一规则限制了机器选择，命题 2.8 的论证不再适用。没有维护时，同质机器可以互换，经典的列表调度论证保证这种规则不丢失最优解；但各机器的维护窗口不同，机器不再可以互换，本文**没有证明**该解码器的序列空间一定包含最优解。作为经验验证，在 $n=3$、$m=2$ 的 16 个单窗口与 16 个多窗口算例上，对两种维护方式（共 64 种组合）穷举全部序列，所得最好值都等于 CP-SAT 证得的最优值（回归测试 `test_both_construction_spaces_contain_a_proven_optimum`）。这只说明在这些小算例上没有发现反例，不能推广为一般结论。

2.6　约束规划模型

小规模算例的精确参考解由 OR-Tools CP-SAT [4] 求得，模型与 (JM)/(FM) 一一对应：

- 每个工件 j 有整数变量 $S_j, C_j \in [0, H]$；对每台机器 i 建一个可选区间 $I_{ji} = [S_j, C_j)$，长度 p_j，存在当且仅当 $x_{ji} = 1$；用 `ExactlyOne` 表示式 (2.5)。
- 每个时间窗 (i,k) 建一个**必选**区间，开始变量的取值范围为 $[L_{ik}, U_{ik} - q_{ik}]$，即式 (2.8)；固定方式另加 $M_{ik} = f_{ik}$。维护区间不能设为可选，否则求解器可以删掉维护。
- 每台机器对其所有存在的加工区间与维护区间施加 `NoOverlap`，即式 (2.9)、(2.10)。
- $T_j \in [0, H]$，用 `AddMaxEquality` 表示 $T_j = \max(0, C_j - d_j)$；目标为 $\min \sum_j T_j$。

由推论 2.6，取值范围 $[0, H]$ 不损失最优值。

对求解结果做以下核对后才接受：

1. 从解中取出分配与时刻，重新构造调度，并独立重算总延期 T；
2. 求解器给出的目标值 z 与下界 b 必须为整数，且满足 $0 \le b \le T \le z$；状态为 `OPTIMAL` 时还要求 $b = T = z$；
3. 调度必须通过第 2.7 节的合法性检查。

只有状态为 `OPTIMAL` 时才称最优解；状态为 `FEASIBLE` 时只称限时可行解，其 T 只是最优值的上界。报告中使用重算的 T，而不是求解器的目标值。在极小算例（$n \le 5$、$m \le 3$）上，CP-SAT 的最优值还与一个独立的穷举程序核对一致。

2.7　调度的合法性条件

所有算法输出的调度都要经过独立的合法性检查器，检查失败就报错，不返回非法调度。检查器不修复调度，逐条检查：

1. **生产部分**（复用 Phase 1 的静态检查器）：每个工件恰好加工一次（式 (2.5)），机器编号合法，同一机器上的加工不重叠（式 (2.9)），$C_j - S_j = p_j$ 且与实例数据一致（式 (2.6)）；
2. **维护部分**：每个时间窗恰好一条维护记录（不能漏、不能重复），时长等于 q_{ik}，整个区间在 $[L_{ik}, U_{ik})$ 内（式 (2.8)），固定方式下开始时刻等于 f_{ik}（式 (2.12)），与本机器上的加工不重叠（式 (2.10)）；
3. **目标**：报告的总延期必须等于按 $\sum_j \max(0, C_j - d_j)$ 独立重算的值（式 (2.7)）。

所有区间按左闭右开处理，所有时间必须是整数。完工时间 $C_{\max}$ 只按生产记录计算，可能早于最后一次维护的结束时刻，这不影响总延期目标。

2.8　算例

下面两个小例子可以手工验证，用来说明联合维护的作用。图中“维”表示维护。

2.8.1　单机例子

两个工件 $p = (2, 2)$，$d = (2, 4)$；一台机器，维护时长 $q = 2$，时间窗 $[0, 6)$，固定方式 $f = 0$。

- 固定维护：维护占用 $[0, 2)$。顺序 A、B 时 A 在 $[2, 4)$、B 在 $[4, 6)$，延期 $2 + 2 = 4$；顺序 B、A 时延期 $0 + 4 = 4$。插入空闲只会推迟完工，所以最优总延期为 4。
- 联合维护：A 在 $[0, 2)$、B 在 $[2, 4)$、维护在 $[4, 6)$，总延期为 0，达到下界。

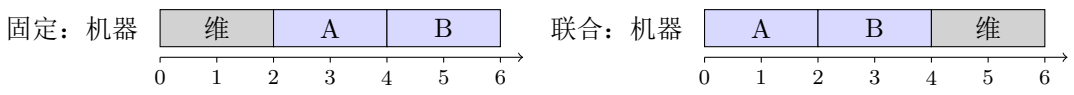


图 2.1: 单机例子：固定维护总延期 4，联合维护总延期 0

2.8.2　两机例子

三个工件 $p=(2,2,2)$，$d=(2,2,4)$；两台机器的维护时长都是 2，时间窗都是 $[0,6)$，固定方式 $f=(0,0)$。

- 固定维护：两台机器都在 $[0,2)$ 维护，之后三个工件的完工时刻至少为 4、4、6，总完工时间至少为 14；交货期之和为 8，而 $\sum_j T_j \geq \sum_j (C_j - d_j)$，所以总延期至少为 6。A 在机器 1 的 $[2,4)$、C 在机器 1 的 $[4,6)$、B 在机器 2 的 $[2,4)$ 达到 6，即为最优。
- 联合维护：机器 1 加工 A $[0,2)$、C $[2,4)$，然后维护 $[4,6)$；机器 2 加工 B $[0,2)$，然后维护 $[2,4)$。三个工件都不延期，总延期为 0。此时生产完工时间 $C_{\max}=4$，最后一次维护在 6 结束。

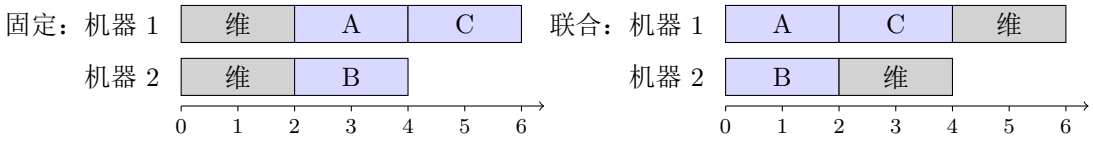


图 2.2: 两机例子：固定维护总延期 6，联合维护总延期 0

两个例子在两种维护方式下的最优值（4 与 0、6 与 0）均已由 CP-SAT（状态 `OPTIMAL`）和独立穷举程序复核一致。检查器另有回归测试，确认漏维护、重复维护、维护越出时间窗、维护与加工重叠、维护时长不符、固定时刻不符、生产部分非法和总延期报错等非法调度都会被拒绝。

2.9　本章小结

本章建立了带周期性预防维护时间窗的同质平行机总延期调度模型，给出了固定维护与联合维护两种方式，以及单窗口这一特例。主要结论：

1. 固定维护的可行域是联合维护的子集，联合最优总延期不大于固定最优总延期（命题 2.3）；
2. 总存在左移的最优解，所有时刻都不超过 $H=U^*+P$（引理 2.5、推论 2.6）；
3. 问题即使只有一台机器也是 NP 难的，机器数作为输入时是强 NP 难的（定理 2.7）；
4. 逐机器追加的构造过程不会陷入死局，且其构造空间包含最优解（命题 2.8）；强化学习所用的“完工最早机器”解码器是否也包含最优解尚未证明，只在小算例上穷举验证过。

这些结论依赖本章的假设：同质机器、单工序、0 时刻到达、正则目标、不同机器维护互不制约。若引入释放时间、工艺路线、换型、共享维修人员或非正则目标，需要重新论证。

参考文献

[1] C.-Y. Lee, Z.-L. Chen. Scheduling jobs and maintenance activities on parallel machines. *Naval Research Logistics*, 2000, 47(2): 145–165.

[2] J. Du, J. Y.-T. Leung. Minimizing total tardiness on one machine is NP-hard. *Mathematics of Operations Research*, 1990, 15(3): 483–495.

[3] M. R. Garey, D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.

[4] L. Perron, F. Didier. CP-SAT. Google OR-Tools, `https://developers.google.com/optimization`.